

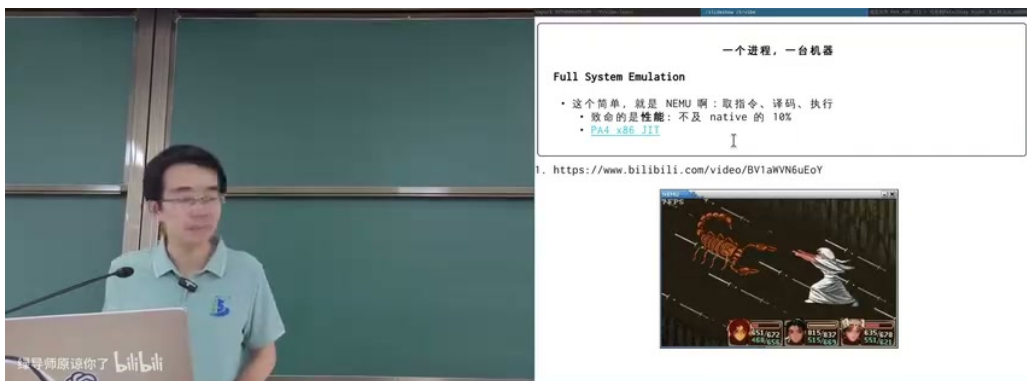
课程笔记

29 - 虚拟机和容器

2026 南京大学操作系统原理

lecture-to-notes

2026-06-20



视频作者/频道：南京大学操作系统原理，蒋炎岩

发布日期：本地课程视频

视频时长：01:23:54

视频链接：未填写，本笔记基于本地视频文件与清洗字幕生成

目录

1 问题入口：虚拟化到底虚拟什么	3
1.1 从状态机看虚拟化	3
1.2 为什么性能决定命运	4
1.3 本章小结	4
2 从解释器到动态翻译	4
2.1 基本块与 JIT	4
2.2 QEMU 的动态生成器	5
2.3 解释、模板翻译与 JIT 的比较	6
2.4 本章小结	6
3 全系统虚拟化的黄金时代	6
3.1 从 Disco 到 VMware	6
3.2 超卖与商业动机	7
3.3 研究品味：做出会改变路线的系统	7
3.4 本章小结	8
4 x86 高性能虚拟机：让多数指令本地执行	8
4.1 普通指令与危险指令	8
4.2 Shadow Page Table	9
4.3 硬件虚拟化与 EPT	9
4.4 虚拟化带来的系统能力	10
4.5 本章小结	10
5 容器路线：让操作系统虚拟化自己	11
5.1 为什么不复制整个操作系统	11
5.2 chroot：文件系统视图的早期隔离	12
5.3 PID 是一个泄漏抽象	12
5.4 本章小结	12
6 Linux namespaces、cgroups 与 Docker	13
6.1 namespaces：隔离名字	13
6.2 cgroups：限制资源	14
6.3 OverlayFS 与镜像分层	15
6.4 容器安全边界	15
6.5 本章小结	15

7	Kubernetes：把容器纳管成云	16
7.1	从“跑一个容器”到“声明一个服务”	16
7.2	调度是多维装箱问题	16
7.3	服务发现、路由与自愈	17
7.4	本章小结	17
8	技术史、研究品味与系统直觉	17
8.1	从 VMware 到 Docker：路线不是必然的	17
8.2	Benchmark crime 与研究训练	17
8.3	品味来自见识，也来自基本功	17
8.4	本章小结	18
9	总结与延伸	18
9.1	讲者的收束	18
9.2	核心机制压缩	18
9.3	带走的三句话	18
9.4	拓展阅读	19
9.5	本章小结	19

1 问题入口：虚拟化到底虚拟什么

本讲从一个看似熟悉的问题开始：我们已经在 NEMU/AM 中体验过“full system emulation”，即把一个硬件状态机完整地解释出来。概念模型很直接：取指、译码、执行、更新状态，必要时模拟设备和特权指令。但真正困难的不是能不能跑，而是能不能快到足以作为基础设施使用。

两条主线

本讲的两条技术主线是：

1. **全系统虚拟化**：把一台机器伪装成多台机器，让未修改的操作系统以为自己独占硬件。
2. **操作系统级虚拟化**：不再复制一整个操作系统，而是在同一个内核中隔离名字空间、资源配额和文件系统视图。

前者通向 VMware、KVM、EPT、VirtIO 等技术；后者通向 chroot、FreeBSD Jail、Linux namespaces、cgroups、OverlayFS、Docker 和 Kubernetes。

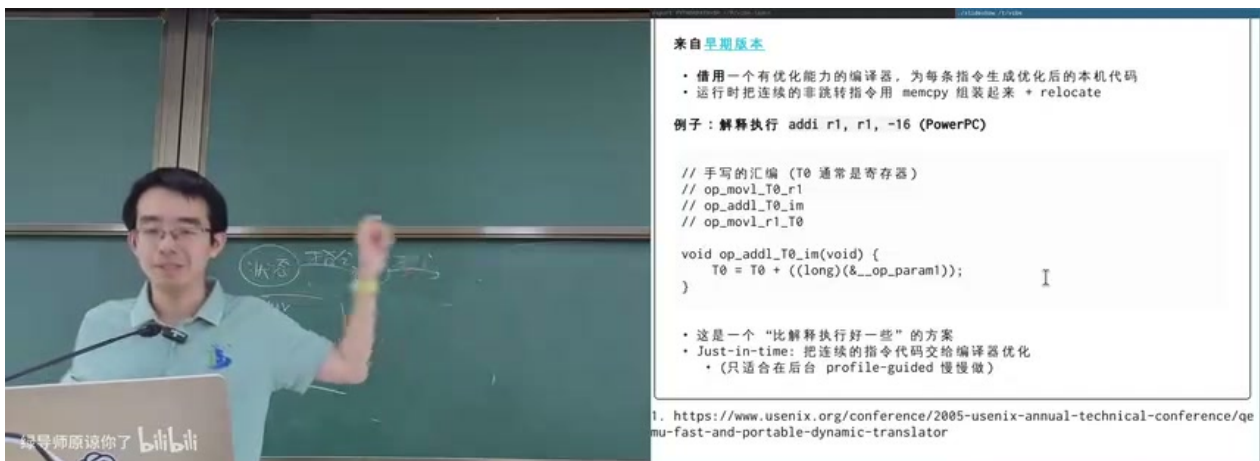


图 1: 从 NEMU/AM 经验切入：虚拟机的概念并不难，性能才是核心问题。¹

1.1 从状态机看虚拟化

CPU、内存和设备都可以看成状态机：指令改变 CPU 状态，内存读写改变内存状态，I/O 指令改变设备状态。如果我们能记录并维护这些状态，就能在宿主机上重现一台客户机的执行。最朴素的模拟器逐条解释指令：

```
1 while (running) {
2     inst = fetch(cpu.pc);
3     decoded = decode(inst);
```

¹视频画面时间区间：00:08:20–00:08:25。

```
4     execute(decoded, &cpu, memory, devices);  
5 }
```

Listing 1: 解释执行的抽象循环

这种循环容易写、容易调试、可移植性高，但每条客户机指令都要付出取指、译码、分派、间接调用等开销。课堂中提到，如果一个程序在模拟器中能达到真机 1/10 的性能已经相当不错；但对于图形游戏或桌面系统，这样的损失常常不可接受。

1.2 为什么性能决定命运

虚拟机不是单纯的教学玩具。它如果要成为基础设施，必须让用户相信“我买到的是台机器”，而不是“我买到的是一台慢十倍的玩具机器”。这要求虚拟机在绝大多数普通指令上接近本地执行速度，只在危险操作上付出额外开销。

$$T_{\text{emu}} = N \cdot (C_{\text{decode}} + C_{\text{dispatch}} + C_{\text{execute}})$$

符号说明：

- N ：客户机指令条数。
- C_{decode} ：每条指令译码成本。
- C_{dispatch} ：解释器分派、函数调用、分支预测失败等成本。
- C_{execute} ：真正更新模拟状态的成本。

虚拟化优化的基本方向就是减少前两项：不要每次都重新译码，不要每条指令都在解释器里分派。

1.3 本章小结

虚拟化的概念来自“everything is a state machine”，但工程难点在于性能与隔离的同时满足。全系统虚拟化追求让未修改操作系统直接运行；容器路线则追求在同一个内核中切出多个受控的执行世界。

2 从解释器到动态翻译

2.1 基本块与 JIT

解释器的第一层优化是利用基本块。一个基本块是没有中途跳转、只有末尾可能跳转的一段连续指令。只要进入基本块，执行顺序就确定，因此可以把这段客户机指令批量翻译成宿主机代码。

JIT 的阈值思想

即时编译不应盲目编译每一个基本块。启动代码、错误路径、一次性初始化路径可能只执行一次，为它们调用编译器会得不偿失。常见策略是先解释执行，同时统计基本块热度；当计数超过阈值，才把它编译成宿主机代码并缓存。

用一个简化成本模型表达：

$$T_{\text{jit}} = T_{\text{warmup}} + T_{\text{compile}} + M \cdot C_{\text{native}}$$

符号说明：

- T_{warmup} ：解释执行和热度统计阶段的成本。
- T_{compile} ：把热基本块翻译成本地代码的成本。
- M ：编译后再次执行该基本块的次数。
- C_{native} ：本地代码执行一次的成本。

只有当 M 足够大，编译成本才能被摊销。

2.2 QEMU 的动态生成器

课堂中特别强调了 QEMU 早期论文 “Fast and Portable Dynamic Translator” 的聪明之处：它不一定调用外部 GCC 编译每个基本块，而是预先为微操作生成模板，再在运行时拷贝模板并 patch 立即数或重定位字段。

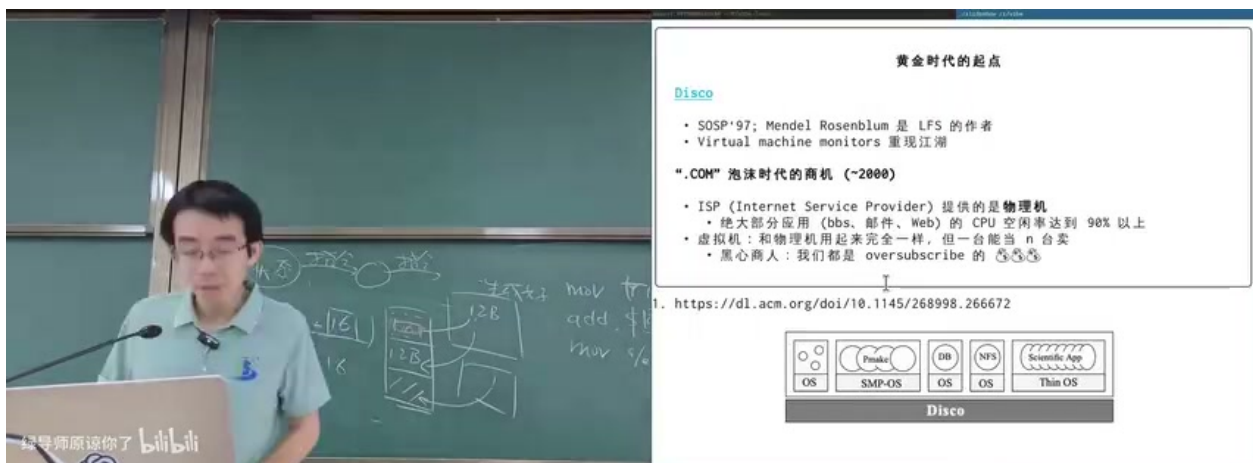


图 2: QEMU 动态翻译：把客户机指令拆成宿主机可复用的模板片段，再用轻量 patch 完成基本块生成。²

以一条客户机指令 `ADDI R1, R1, 16` 为例，翻译器可以把它拆成三个微操作：

²视频画面时间区间：00:15:40–00:15:45。

```

1 T0 = guest_reg[R1];
2 T0 = T0 + IMM;      // IMM is patched at translation time
3 guest_reg[R1] = T0;

```

Listing 2: 一条带立即数指令的微操作分解

第一次构建翻译器时，模板已经由 C 编译器为当前宿主体系结构生成。运行时只需把模板拷贝到基本块缓存，并把 IMM 之类的占位符改成客户机指令里的实际数值。这让翻译成本接近“内存拷贝 + 少量 patch”，介于逐条解释和完整 JIT 编译之间。

为什么这很系统

这类动态翻译把编译器、链接、重定位、代码缓存、内存保护和指令集语义联系在一起。它不是单独的“编译器优化”，也不是单独的“操作系统技巧”，而是系统软件常见的跨层设计：在正确性边界内移动成本。

2.3 解释、模板翻译与 JIT 的比较

方案	优点	代价	适用场景
逐条解释	简单、可移植、易调试	每条指令重复译码和分派	教学模拟器、冷路径
模板动态翻译	编译成本低，性能显著优于解释	模板设计复杂，优化空间受限	QEMU 早期 TCG 思路
JIT 编译	热路径接近本地代码性能	编译器复杂，热度判断重要	JVM、V8、现代语言 VM

2.4 本章小结

从解释器到动态翻译，本质是把“每次执行都做的工作”移到“翻译时做一次”。基本块、热度阈值、模板、重定位和代码缓存共同构成了高性能模拟器的基础。但即使如此，全系统虚拟化还有更大的机会：同 ISA 的客户机是否可以直接在 CPU 上跑？

3 全系统虚拟化的黄金时代

3.1 从 Disco 到 VMware

1990 年代末，Mendel Rosenblum 等人在 Disco 中重新激活了 1970 年代的 Virtual Machine Monitor 思想：在一台机器上运行多个未修改的操作系统。课堂中强调，这不只是论文问题，也恰好遇到了互联网泡沫时期的数据中心需求。

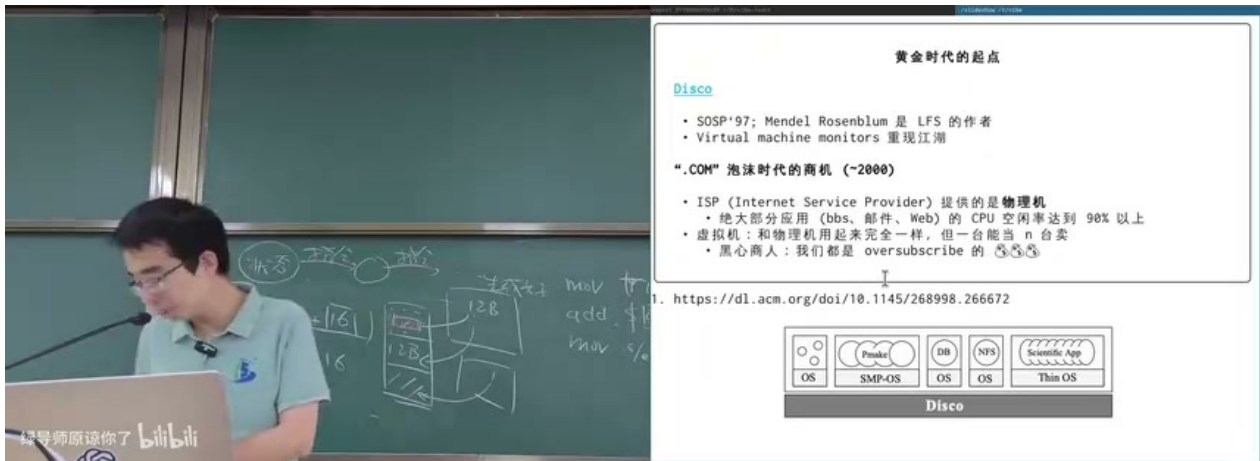


图 3: Disco 与互联网时代的起点：一台物理机可以被包装成多台可售卖的虚拟机。³

当时的 ISP 或数据中心面临一个明显机会：许多 Web、BBS、邮件服务长期 CPU 空闲率很高。如果只按物理机出租，资源利用率低；如果把一台机器切成多台虚拟机出售，就可以让多个客户共享同一台物理服务器，同时保持“像独占机器一样”的体验。

3.2 超卖与商业动机

虚拟化的商业逻辑可以用资源利用率表达：

$$U = \frac{\sum_i r_i(t)}{R_{\text{physical}}}$$

符号说明：

- $r_i(t)$ ：第 i 个租户在时刻 t 实际使用的资源。
- R_{physical} ：物理机器总资源。
- U ：实际资源利用率。

如果每个客户只偶尔达到峰值，那么云厂商可以承诺总量超过物理总量的虚拟资源，也就是 oversubscription。只要不同客户的峰值不同时出现，服务体验仍然可接受。

超卖不是魔法

超卖依赖统计假设。只要所有租户同时进入高负载，或者某些租户通过旁路消耗共享资源，性能隔离就会失效。因此虚拟化系统必须同时解决性能、隔离、计费 and 故障处理，而不是只把机器“切小”。

3.3 研究品味：做出会改变路线的系统

课堂中有一段很重要的反思：好的系统论文不是简单报告“我比别人快一点”，而是把一个旧想法在新条件下真正做成。Disco 和 VMware 的意义就在于，它们把早期大型机时

³视频画面时间区间：00:18:40–00:18:45。

代的虚拟机想法带入了 PC 服务器和互联网时代。

3.4 本章小结

全系统虚拟化的黄金时代来自技术与商业的交汇：同一台服务器被虚拟成多台可管理、可迁移、可售卖的机器。这个方向最终深刻影响硬件设计、云计算基础设施和数据中心管理。

4 x86 高性能虚拟机：让多数指令本地执行

4.1 普通指令与危险指令

如果客户机和宿主机使用同一 ISA, 最诱人的想法是：让绝大多数普通指令直接在 CPU 上执行，只拦截那些会破坏全局状态的操作。例如算术、内存中的数据结构遍历、函数调用等，本来就可以用本地速度完成；困难在于系统调用、特权指令、页表切换、I/O 等。

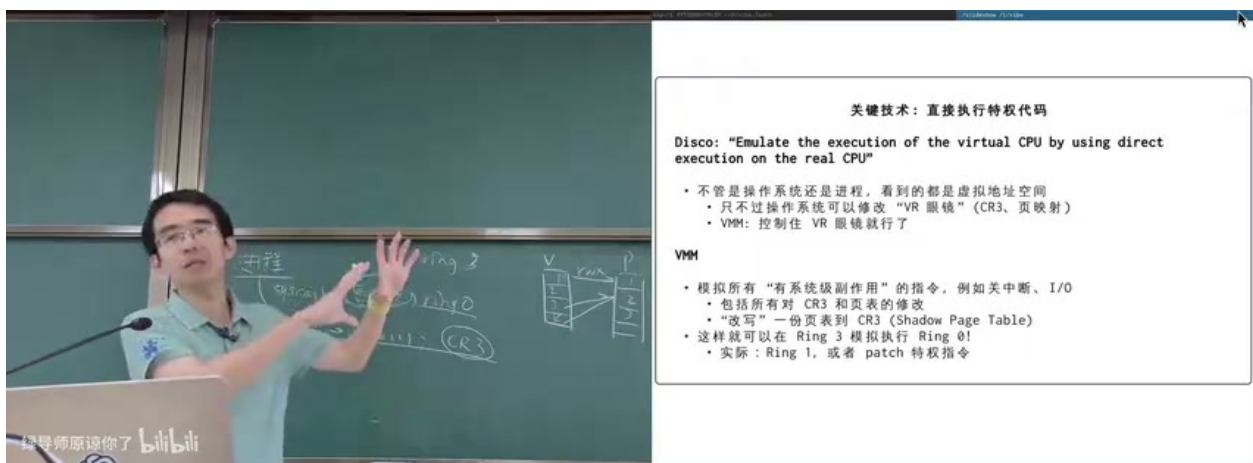


图 4: 进程和内核的特权级差异：Ring 0 指令不能随便交给客户机直接执行。⁴

在 x86 传统模型中，应用运行在 Ring 3，内核运行在 Ring 0。客户机操作系统如果也直接运行在真实 Ring 0，就能关中断、改 CR3、访问设备端口，进而破坏宿主机。VMware 的早期技巧是把客户机内核放到较低权限的 Ring 1，让危险指令 trap 到 VMM，由 VMM 模拟其效果。

Trap-and-emulate 的核心

客户机的大多数普通代码直接执行；凡是可能改变真实硬件全局状态的操作，都必须 trap 出来，由虚拟机监控器根据客户机看到的虚拟硬件状态进行模拟。

⁴视频画面时间区间：00:43:30–00:43:35。

4.2 Shadow Page Table

页表是全系统虚拟化中最难的一部分。客户机内核以为自己拥有“物理页 1、2、3”，但这些只是客户机物理页；宿主机真正拥有的是机器物理页。VMM 必须维护两层映射：

层次	映射	谁维护
客户机页表	Guest virtual page → Guest physical page	客户机操作系统
机器分配表	Guest physical page → Host machine page	VMM/Hypervisor
影子页表	Guest virtual page → Host machine page	VMM 综合生成

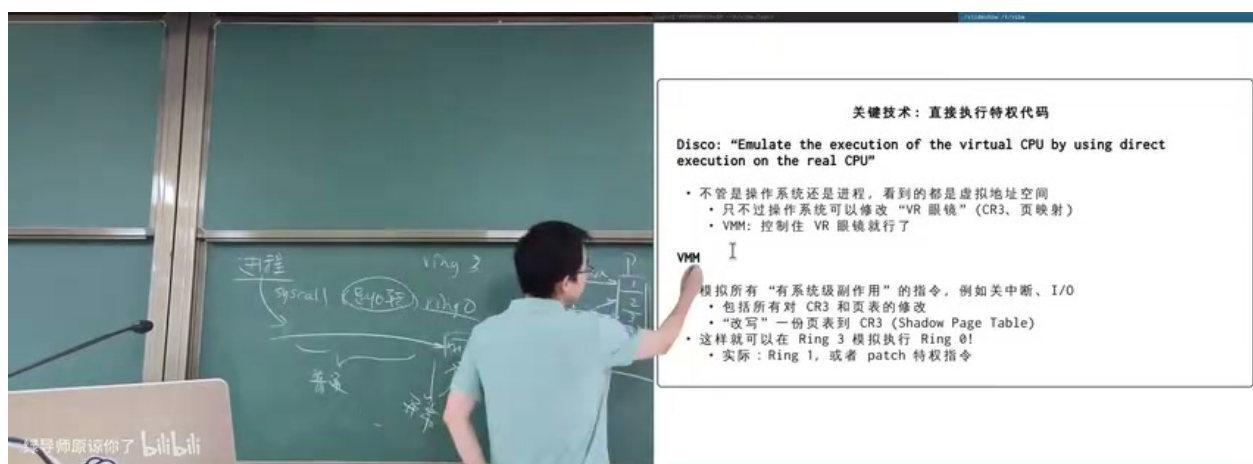


图 5: 影子页表：客户机声称的物理页被 VMM 重新映射到真实机器页。⁵

当客户机执行 `mov` 到 CR3、修改页表项或刷新 TLB 时，VMM 不能让它直接生效，而要更新对应的 shadow page table，并把真正的 CR3 指向 VMM 构造好的页表。这样 CPU 看到的是安全的真实映射，客户机看到的仍然是自治的虚拟物理内存。

4.3 硬件虚拟化与 EPT

软件 VMM 的成功反过来影响了硬件。Intel VT-x/AMD-V 提供 VMX root/non-root 或类似模式，使客户机可以在受控的虚拟机模式下运行；危险事件触发 VM exit。后来 EPT/NPT 把第二层地址翻译也交给硬件，从而减少 shadow page table 的软件维护成本。

⁵视频画面时间区间：00:48:10–00:48:15。

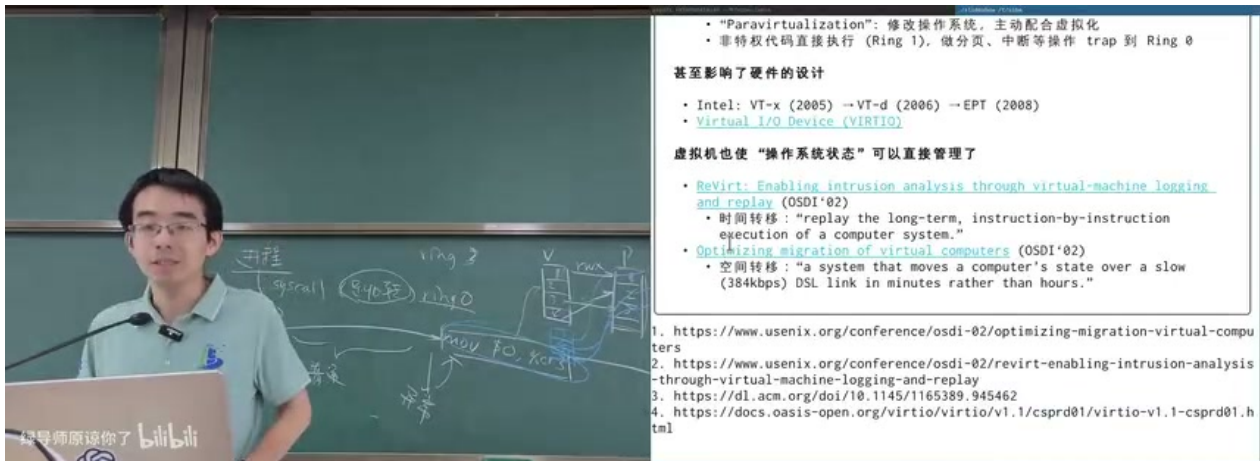


图 6: 硬件虚拟化与 EPT: 从软件 Ring hack 走向处理器直接支持 VM exit 和两级地址翻译。⁶

如果客户机使用四级页表, EPT 再提供四级从 guest physical 到 host physical 的映射, 硬件一次访存可能经历更深的页表遍历。但这换来了更清晰的隔离边界, 以及更少的软件 trap 和页表同步。

4.4 虚拟化带来的系统能力

一旦整个机器被虚拟成一个对象, 很多能力自然出现:

- **VirtIO**: 用标准化半虚拟设备接口减少模拟真实硬件的复杂度。
- **record/replay**: 记录 VM exit、设备输入和中断时序, 理论上可以重放整台机器的执行。
- **migration**: 暂停虚拟机, 复制内存与设备状态, 在另一台机器恢复执行。
- **sandbox**: 把不可信工作负载放进可销毁、可快照、可重放的机器边界中。

4.5 本章小结

高性能全系统虚拟化的关键是“多数指令本地执行, 少数危险事件受控退出”。Ring 技巧、binary translation、shadow page table、VM exit、EPT 和 VirtIO 都服务于这个目标: 让客户机相信自己独占硬件, 同时让宿主机保持最终控制权。

⁶视频画面时间区间: 00:49:35-00:49:40。

5 容器路线：让操作系统虚拟化自己

5.1 为什么不复制整个操作系统

全系统虚拟机的隔离强，但成本也高。课堂中提出一个反问：我们真正想运行的是应用程序，而应用程序和操作系统交互的唯一正式入口是系统调用。那么，为什么不能让一个内核同时提供多个隔离的系统调用世界？



图 7: OSID 思维实验：给内核对象加一维身份标记，就能把一个内核切成多个相互不可见的世界。⁷

想象在 xv6 的所有内核对象上增加一个 osid 字段：

```
1 struct proc {
2     int pid;
3     int osid;
4     ...
5 };
6
7 bool visible(struct proc *viewer, struct proc *target) {
8     return viewer->osid == target->osid;
9 }
```

Listing 3: OSID 的极简想象模型

这样同一个系统中可以有多个“PID 1”的语义，或者至少有多个互相不可见的进程集合。文件、挂载点、IPC、网络端口、用户 ID、主机名、时间视图等对象也可以采用类似思路。

⁷视频画面时间区间：00:56:25-00:56:30。

容器的本质

容器不是一台轻量虚拟机，而是同一个内核中的一组隔离视图加资源控制。它没有复制内核，只复制或隔离了应用可见的操作系统对象。

5.2 chroot：文件系统视图的早期隔离

Version 7 UNIX 的 `chroot` 已经提供了一个早期思路：把进程的根目录切到某个子目录，使它看到一个不同的文件系统根。但只有 `chroot` 不够，因为 PID、网络、IPC、用户和资源配额仍然共享。它更像容器的一块拼图，而不是完整容器。

5.3 PID 是一个泄漏抽象

课堂中特别补充了 PID 的问题：PID 会回收复用。一个进程先通过 `ps` 找到目标 PID，过了一段时间再 `kill`，此时那个 PID 可能已经属于另一个进程。单纯整数 ID 不能表达稳定引用。

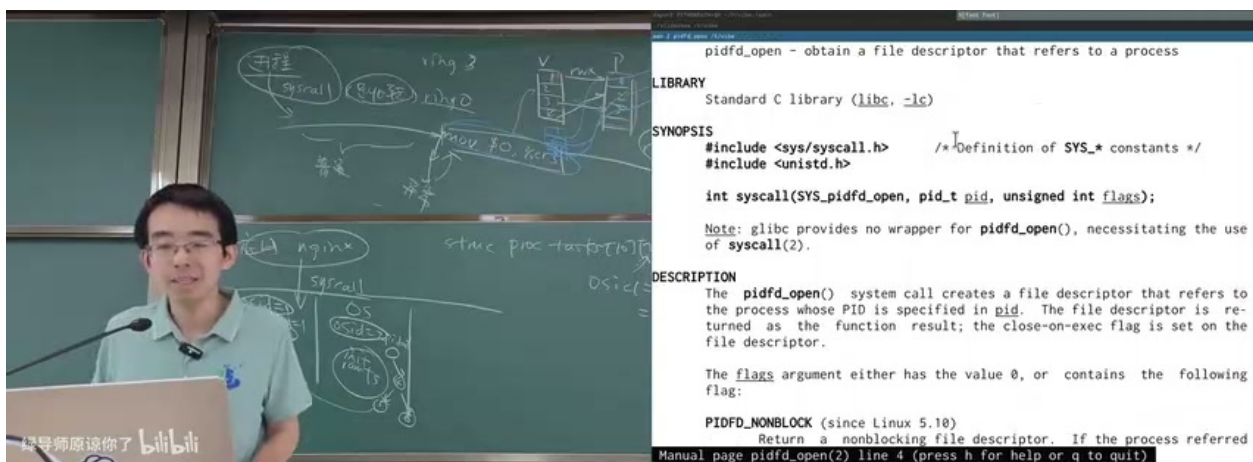


图 8: `pidfd_open`：用文件描述符持有进程引用，避免 PID 复用导致误杀。⁸

`pidfd_open` 的设计看似奇怪，却符合 UNIX 的对象模型：引用一个内核对象最好通过文件描述符。文件描述符携带引用计数，能防止对象在使用期间被错误复用。

隔离与引用是两件事

`namespace` 解决“我能看到谁”，`pidfd` 解决“我引用的是不是同一个对象”。容器系统如果只考虑命名空间，不考虑稳定引用和生命周期，也会留下竞态条件。

5.4 本章小结

容器路线把虚拟化问题从“模拟硬件”改写为“隔离内核对象视图”。OSID 思维实验说明，只要系统调用能按照租户身份过滤对象，单个内核就可以呈现多个近似独立的操作系统

⁸ 视频画面时间区间：00:59:55–01:00:00。

统世界。

6 Linux namespaces、cgroups 与 Docker

6.1 namespaces：隔离名字

Linux namespaces 把不同类型的内核对象放进不同名字空间。课堂中列举了 user、mount、PID、IPC、network、UTS、time、cgroup 等多个维度。一个进程可以共享某些 namespace，同时拥有另一些独立 namespace。

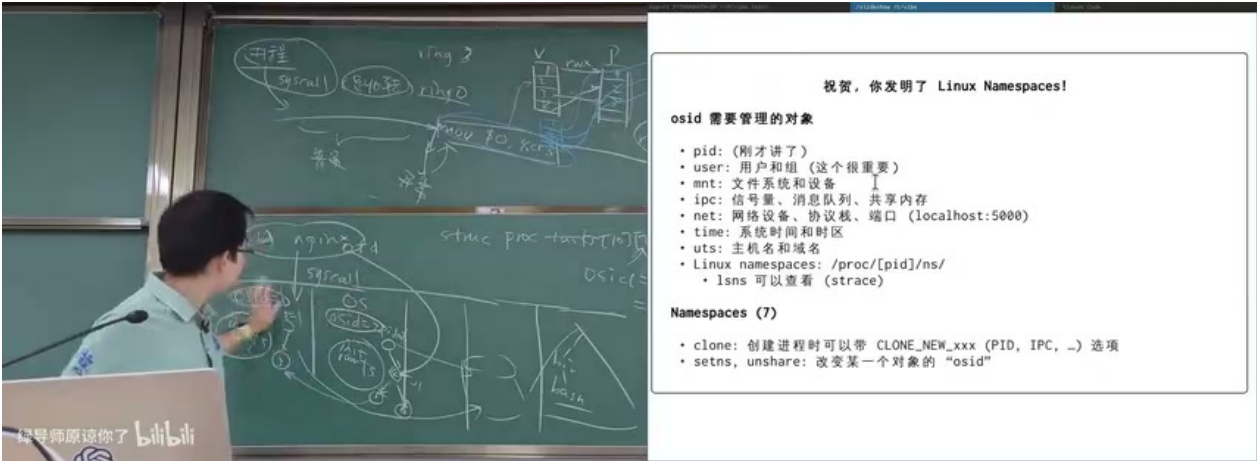


图 9: Linux namespaces：从 PID、用户、挂载点到网络端口，分别隔离应用可见的对象名字。⁹

namespace	主要隔离内容
PID	进程编号空间；不同容器可看到自己的 PID 1
Mount	挂载点集合；容器拥有独立文件系统视图
User	UID/GID 映射；容器内 root 不等于宿主机 root
Network	网卡、路由表、端口、localhost 视图
IPC	System V IPC、POSIX message queue 等
UTS	主机名与域名
Time	时间偏移视图
Cgroup	进程看到的 cgroup 层次

⁹视频画面时间区间：01:05:00–01:05:05。

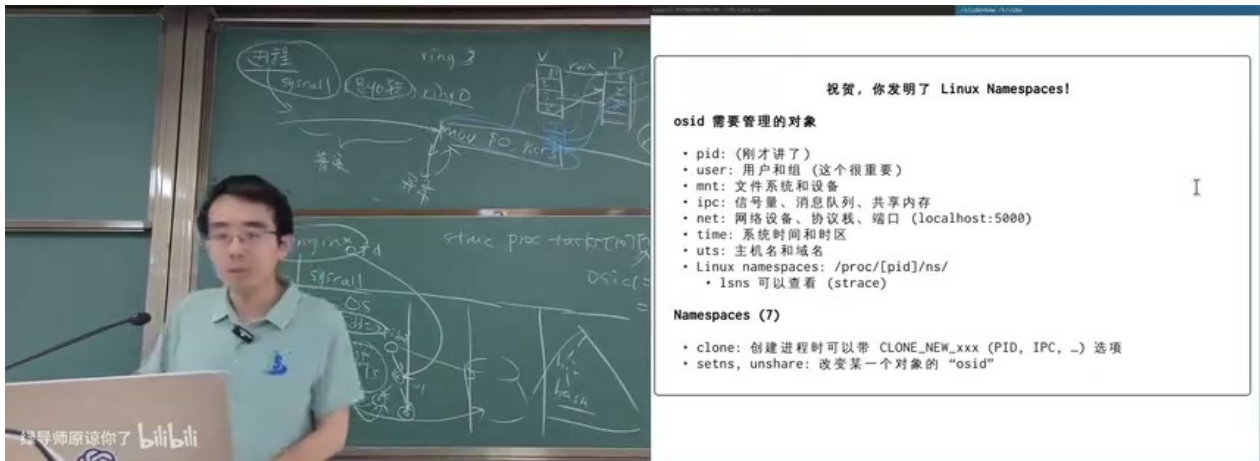


图 10: `/proc/<pid>/ns` 与 `lsns`: namespace 在 Linux 中表现为可观察、可比较的内核对象。¹⁰

从 `/proc/<pid>/ns` 可以看到某个进程所在的 namespace 标识。两个进程如果某个 namespace inode 相同, 就共享那类视图; 否则它们在那一类对象上被隔离。

6.2 cgroups: 限制资源

只有隔离名字还不够。黑心商人真正想售卖的是可计量、可控制的资源份额。因此必须限制 CPU、内存、I/O、进程数、网络等资源使用。cgroups 的目标不是隐藏对象, 而是把一组进程放入控制组, 并对它们施加策略。

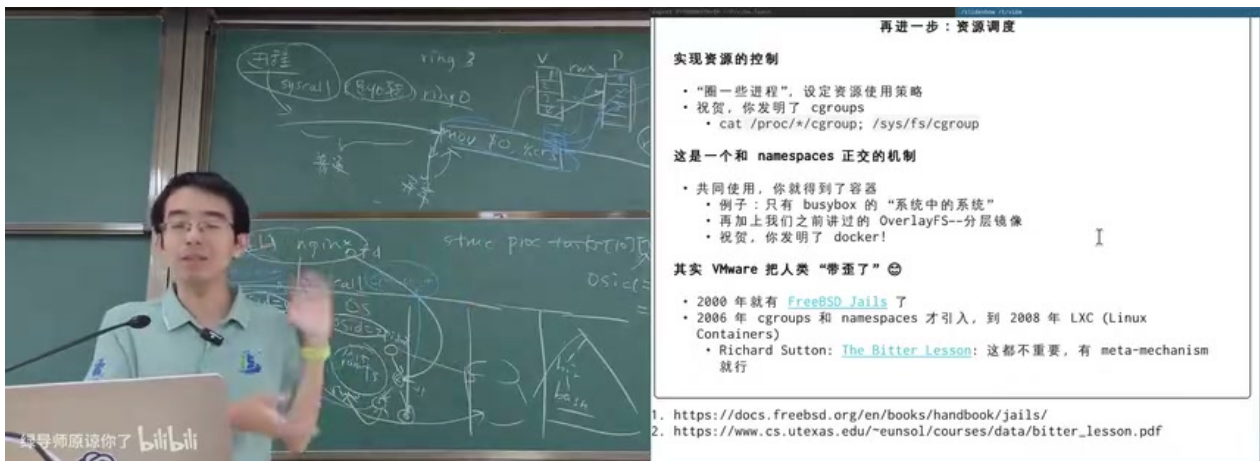


图 11: cgroups 与 OverlayFS: 资源控制和分层文件系统共同补齐容器的商业可用性。¹¹

cgroups v2 采用统一层次结构。每个节点可以配置控制器, 例如 CPU 权重、内存上限、I/O 限速、进程数量上限。调度器和资源管理器根据这些控制器进行计量和限制。

¹⁰ 视频画面时间区间: 01:08:50–01:08:55。

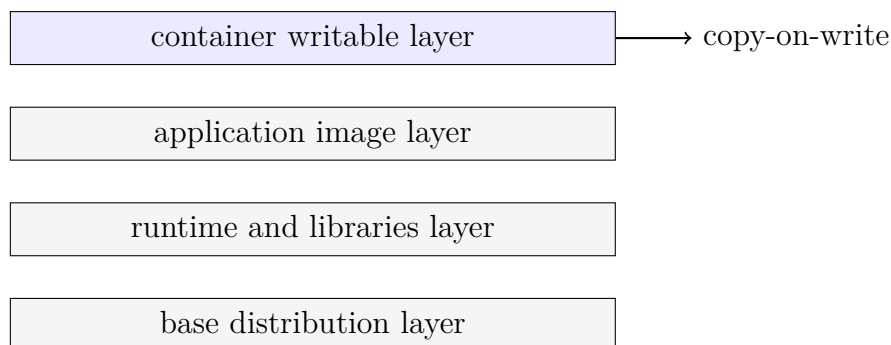
¹¹ 视频画面时间区间: 01:11:10–01:11:15。

namespace 与 cgroup 正交

namespace 回答“你看见什么名字”，cgroup 回答“你能用多少资源”。一个容器通常同时使用二者：前者提供隔离感，后者提供配额和计费基础。

6.3 OverlayFS 与镜像分层

Docker 的另一个关键拼图是分层文件系统。多个容器可以共享只读基础镜像，只在上层记录自己的写入。这让启动大量相同镜像的容器变得很便宜：



这解释了容器为什么能“见缝插针”：它不需要为每个实例复制完整系统盘，只需要共享基础层并增加很薄的可写层。

6.4 容器安全边界

容器比虚拟机轻，但安全边界更薄。虚拟机有独立客户机内核，逃逸需要突破 VMM 或设备模拟层；容器共享宿主机内核，一旦内核 namespace、权限检查或 cgroup 控制器存在漏洞，租户可能越权访问对象或造成拒绝服务。

容器不是安全银弹

容器适合高密度部署和快速启动，但不要把它误认为等同于硬件虚拟机的隔离。实际系统常把容器、虚拟机、seccomp、capabilities、LSM、只读根文件系统等机制组合起来。

6.5 本章小结

Linux 容器由多个机制拼合而成：namespaces 隔离名字，cgroups 限制资源，OverlayFS 提供分层镜像，capabilities/seccomp 等收缩权限。Docker 的成功不是单点发明，而是把这些内核能力包装成可复现、可分发、可运维的工程接口。

7 Kubernetes：把容器纳管成云

7.1 从“跑一个容器”到“声明一个服务”

如果一台机器上有几个容器，手工启动即可；如果多个数据中心里有数十万容器，就需要更高层的编排系统。Kubernetes 的核心思想不是让用户指定“在哪台机器上运行哪个进程”，而是让用户声明期望状态：某个镜像需要多少副本、暴露什么服务、需要多少资源、如何滚动升级。

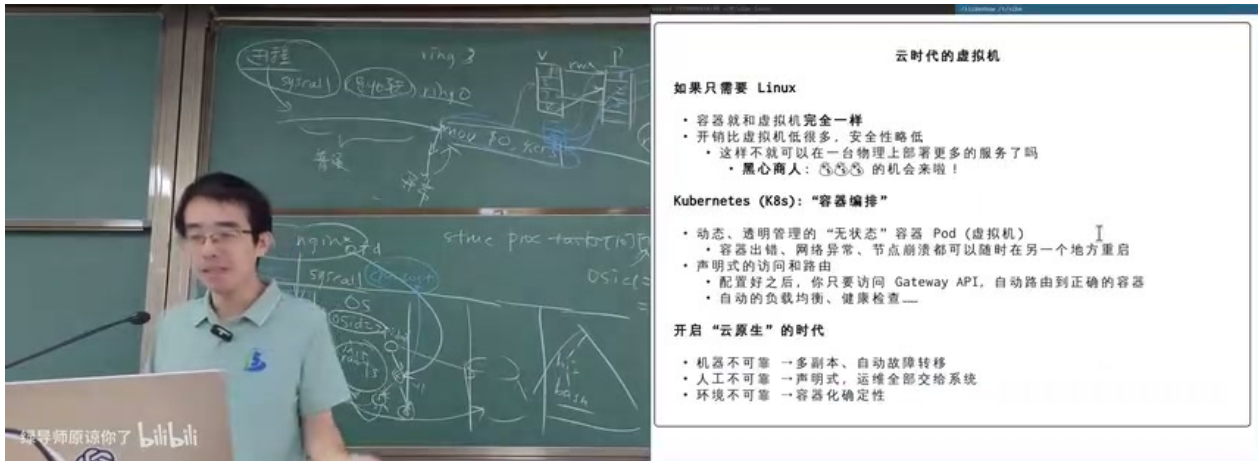


图 12: Kubernetes：在云原生语境中，容器成为可调度、可复制、可替换的无状态单元。¹²

7.2 调度是多维装箱问题

课堂中把容器调度类比为背包问题。每个容器都有 CPU、内存、磁盘、网络、亲和性、反亲和性等需求；每台机器有剩余资源和拓扑位置。调度器希望用尽可能少的机器容纳尽可能多的工作负载，同时保持故障域、延迟和服务质量。

$$\sum_{p \in P_m} d_{p,k} \leq C_{m,k}, \quad \forall m, k$$

符号说明：

- P_m ：被放到机器 m 上的 pod 集合。
- $d_{p,k}$ ：pod p 对第 k 类资源的需求，例如 CPU 或内存。
- $C_{m,k}$ ：机器 m 上第 k 类资源的容量。

这是多维约束优化问题，静态最优通常不可行，因为集群状态不停变化：扩容、缩容、故障、迁移、滚动升级、流量波动都会改变约束。

¹²视频画面时间区间：01:17:00–01:17:05。

7.3 服务发现、路由与自愈

Kubernetes 还提供服务发现和路由抽象。用户访问服务名或 Gateway，而不需要知道请求最终落在哪个 pod。控制面持续比较期望状态和实际状态，如果 pod 死亡、节点失联或健康检查失败，就重新调度。

声明式系统的力量

声明式 API 把“我要什么”与“怎么达到”分开。用户提交期望状态，控制循环负责不断把现实拉回期望状态。这是 Kubernetes 区别于简单脚本集合的核心。

7.4 本章小结

Kubernetes 把容器从单机工具提升为数据中心抽象：pod 是可替换单元，service 是稳定访问入口，scheduler 是资源装箱器，controller 是状态收敛机制。云原生的本质是又一层虚拟化：用户看到服务，平台管理机器。

8 技术史、研究品味与系统直觉

8.1 从 VMware 到 Docker：路线不是必然的

课堂中的一个重要观点是：VMware 的路线并非唯一答案。全系统虚拟化先成功，硬件随后为它增加 VT-x/EPT 等支持；但从另一个角度看，如果早期操作系统级虚拟化更早获得足够工程投入，容器路线也可能更早成为主流。

这说明系统技术的发展不是线性“最优解”，而是由硬件成熟度、商业需求、工程可行性、社区生态和资本激励共同塑造的。

8.2 Benchmark crime 与研究训练

讲者借虚拟化历史提醒大家：真正好的工作往往不是在某个 benchmark 上挤出百分之几提升，而是重新定义问题边界。benchmark 可以帮助评估系统，但也容易诱导 cherry-picking、隐藏不利结果、夸大贡献。

论文与系统的基本诚实

可以性能不好，可以不 scale，可以只适合某个场景，但不要让读者误以为系统具备并不存在的性质。系统研究的可信度来自问题定义、假设边界、实验覆盖和失败案例。

8.3 品味来自见识，也来自基本功

讲者反复强调“看过好东西”的重要性。没有数字逻辑、体系结构、编译、操作系统的基本功，就无法判断一个系统 trick 到底好在哪里；没有足够广阔的技术视野，也容易把局

部小优化误认为大问题。

本讲背后的价值判断

虚拟化和容器不是孤立知识点。它们展示了系统领域最值得学习的能力：把抽象边界画对，把性能成本移走，把隔离边界守住，并在真实需求中找到可以改变世界的杠杆。

8.4 本章小结

技术路线会被时代、商业和生态塑造。学习操作系统不只是记 API 或机制，而是训练一种系统直觉：什么时候该模拟，什么时候该隔离，什么时候该把对象暴露为文件描述符，什么时候该把硬件、内核和用户态协同设计。

9 总结与延伸

9.1 讲者的收束

本讲最后把虚拟化、容器和云原生放回更大的技术史中：财富自由、云计算、开源生态、论文品味、课程实验和个人选择都不是互相无关的。一个系统技术如果抓住了真实需求，就可能从论文、实验室原型变成数据中心基础设施。即使某条路线后来被另一条路线挤压，它也可能留下硬件接口、软件抽象和工程经验。

9.2 核心机制压缩

层次	关键问题	代表机制
指令模拟	如何让异构 ISA 程序跑起来	解释器、basic block、动态翻译、JIT
全系统虚拟化	如何运行未修改操作系统	trap-and-emulate、shadow page table、VM exit、EPT
操作系统级虚拟化	如何让同一内核呈现多个世界	chroot、namespaces、cgroups、OverlayFS
云原生编排	如何管理海量可替换服务	Kubernetes、声明式 API、调度器、控制循环

9.3 带走的三句话

1. **虚拟化不是一种技术，而是一组边界设计。**有时边界在硬件机器外，有时在内核对象外，有时在服务 API 外。
2. **性能来自让常见路径少付钱。**VMware 让普通指令本地执行，QEMU 摊销译码成本，Docker 共享镜像层，Kubernetes 让机器利用率更高。

3. **隔离必须和资源控制一起设计。**看不见对象不等于用不了资源；能限制资源也不等于安全隔离完备。

9.4 拓展阅读

- Disco: Running Commodity Operating Systems on Scalable Multiprocessors.
- QEMU: A Fast and Portable Dynamic Translator.
- Linux manual pages: `namespaces(7)`, `cgroups(7)`, `pidfd_open(2)`.
- Kubernetes documentation: Pods, Services, Deployments, Scheduling.

9.5 本章小结

从 NEMU 到 QEMU，从 Disco 到 VMware，从 chroot 到 Docker，再到 Kubernetes，本讲展示了一条非常系统的主线：虚拟化就是在不同层次上重写“谁拥有机器”这个问题。理解这条线，能帮助我们判断一个系统设计到底是在复制成本、转移成本，还是消除成本。